Q2: UI Design and Advanced UI

** = supplementary, not in the lecture materials · Views and layouts · Toast · Intents · ScrollView · ListView · Adapters · RecyclerView · Fragments · Material Design · UI/UX flow

1. The basic idea: how an Android screen is built

Every Android screen is made of two parts:

- **XML layout file** (in `res/layout/`, for example `activity_main.xml`): describes *what* is on the screen and *where*.
- **Kotlin file** (for example `MainActivity.kt`): describes *what happens* when the user interacts.

An **Activity** is one screen of the app. In `onCreate()` it loads its layout with `setContentView(R.layout.activity_main)`.

Term	Meaning
View	Any single UI element the user sees or touches (TextView, Button, EditText, ImageView).
ViewGroup / Layout	A container that holds other views and decides how they are positioned (LinearLayout, ConstraintLayout). Every layout is a ViewGroup.
Widget	Common name for a ready-made View such as a Button.
R class	Auto-generated class that gives every resource an ID. `R.layout.activity_main` is a layout, `R.id.btnLogin` is a view ID, `R.string.app_name` is a string.
`android:id="@+id/btnLogin"`	Gives a view an ID. The + means “create this ID”.
`match_parent`	View becomes as big as its parent container.
`wrap_content`	View is only as big as its content needs.
dp (density-independent pixels)	Unit for sizes, margins and padding. Looks the same physical size on every screen density.
sp (scale-independent pixels)	Unit for **text size only**. Like dp, but also scales with the user’s font-size setting (accessibility).
padding vs **margin**	Padding = space *inside* the view’s border. Margin = space *outside* the border.

Exam rule: use `dp` for dimensions and spacing, `sp` for text. Never use fixed pixels (`px`), because they look different on different screens.

Common views

View	Use	Key attribute / method
TextView	Shows read-only text	`android:text`, `textView.text = "Hi"`
EditText	Lets the user type input	`android:hint`, `android:inputType="textPassword"`, read with `editText.text.toString()`
Button	Triggers an action	`setOnClickListener { }`
ImageView	Shows an image	`android:src="@drawable/logo"`
Spinner	Drop-down list to pick one value	needs an Adapter
CheckBox / RadioButton / Switch **	On/off and single choice inputs	`isChecked`

Useful `inputType` values: `text`, `textPassword` (hides characters), `textEmailAddress` (shows @ key), `number`, `phone`.

2. Layouts (ViewGroups)

A layout decides how its children are positioned and sized. The right choice affects performance, maintenance and how well the UI adapts to different screens.

Layout	How it arranges children	Best use	Advantages	Disadvantages
LinearLayout	One after another in a single row or column (`android:orientation="vertical"` or `"horizontal"`)	Forms, simple menus, login screen	Simple, beginner-friendly, `layout_weight` splits space	Deep nesting hurts performance
RelativeLayout (legacy)	Relative to siblings or parent (`layout_below`, `alignParentTop`)	Medium screens (older apps)	Avoids some nesting	Needs two measure passes, slower; mostly replaced
ConstraintLayout (recommended)	Each view constrained to parent or other views (`app:layout_constraintTop_toTopOf="parent"`)	Modern, complex, responsive screens	Flat hierarchy, best performance; chains, guidelines, barriers, ratios	Steeper learning curve, verbose XML
FrameLayout	Stacks children on top of each other (later child drawn on top)	Fragment container, image with text overlay, loading spinner	Very lightweight	Not for many separately arranged views
TableLayout (legacy)	Rows (`<TableRow>`) and columns	Spreadsheet-like data entry forms	Structured, intuitive	No cell spanning, less flexible
GridLayout	Fixed grid using `rowCount`/`columnCount`	Calculator, dashboard	Supports row/column spanning	Limited for irregular layouts

Which layout for which screen: Login form → LinearLayout or ConstraintLayout. Registration form → ConstraintLayout. Calculator → GridLayout. Image with text overlay → FrameLayout. Table data entry → TableLayout. Modern responsive app → ConstraintLayout. (New projects in 2026 may also use **Jetpack Compose**, Google's declarative Kotlin UI toolkit that replaces XML.)

layout_weight tip: in a LinearLayout, set the size on the weighted direction to `0dp` and give each child `layout_weight="1"` to make them equal sizes.

Lecture example: Login screen in XML

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <EditText
        android:id="@+id/etUsername"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Username" />

    <EditText
        android:id="@+id/etPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Password"
        android:inputType="textPassword" />

    <Button
        android:id="@+id/btnLogin"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Login" />
</LinearLayout>
```

ConstraintLayout version of one button centred on screen:

```
<Button
    android:text="Login"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

3. Event handling and findViewById

The XML on its own does nothing. Kotlin code must find each view and attach a **listener**.

Event flow (from the slides): User Action → Event Generated → Listener Activated → Code Executed → UI Feedback

- `findViewById<Type>(R.id.someId)` searches the layout and returns the view with that ID, so Kotlin can use it.
- `setOnClickListener { ... }` runs the code inside the braces every time the view is tapped.

```
class LoginActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_login)          // load the XML

        // Step 1: find the views
        val username = findViewById<EditText>(R.id.etUsername)
        val password = findViewById<EditText>(R.id.etPassword)
        val loginButton = findViewById<Button>(R.id.btnLogin)

        // Step 2: react to the click
        loginButton.setOnClickListener {
            val user = username.text.toString()
            val pass = password.text.toString()
            if (user.isNotEmpty() && pass.isNotEmpty()) {
                Toast.makeText(this, "Login Successful", Toast.LENGTH_SHORT).show()
            } else {
                Toast.makeText(this, "Please enter all fields", Toast.LENGTH_SHORT).show()
            }
        }
    }
}
```

Other listeners **: `setOnLongClickListener { ...; true }` (long press, must return true), `setOnCheckedChangeListener` (CheckBox/Switch), `setOnItemClickListener` (ListView rows).

4. Toast messages

A **Toast** is a small, temporary pop-up message at the bottom of the screen. It disappears on its own, cannot be clicked, and does not block the user. Used for quick feedback like “Saved” or “Login successful”.

```
Toast.makeText(this, "Saved successfully", Toast.LENGTH_SHORT).show()
```

Part	Meaning
`Toast.makeText(...)`	Creates the Toast (does not show it yet)
`this`	The **context** (the current Activity). In a Fragment use `requireContext()`. In an adapter use `holder.itemView.context` or `view.context`.
`"Saved successfully"`	The message text
`Toast.LENGTH_SHORT` / `Toast.LENGTH_LONG`	Duration: short (about 2 s) or long (about 3.5 s)
`.show()`	Actually displays it. **Forgetting `.show()` is the most common mistake**: nothing appears.

Show a value inside a Toast with a string template: `Toast.makeText(this, "Hello $name", Toast.LENGTH_LONG).show()`

Feedback type	When to use
Toast	Short, passive info; no user action needed
Snackbar (Material) **	Short message at the bottom that can include an action button such as “UNDO”: `Snackbar.make(view, "Deleted", Snackbar.LENGTH_SHORT).setAction("UNDO") { }.show()`
AlertDialog	When the user must confirm or decide (for example “Do you want to logout?”)

5. Intents: moving between screens

An **Intent** is a messaging object that asks Android to do something, most commonly to open another Activity.

Type	Meaning	Example
Explicit Intent	Names the exact target component (class) inside your app	`Intent(this, HomeActivity::class.java)`
Implicit Intent **	Describes an action; Android finds a suitable app to handle it	`Intent(Intent.ACTION_VIEW, Uri.parse("https://uok.ac.lk"))` opens a browser

```
// Login -> Home, sending data (lecture example)
val intent = Intent(this, HomeActivity::class.java)
intent.putExtra("USERNAME", username)      // key-value data attached to the Intent
startActivity(intent)

// Inside HomeActivity.onCreate(): read it with the SAME key
val username = intent.getStringExtra("USERNAME")
tvWelcome.text = "Welcome $username"

// Go back to the previous screen
btnBack.setOnClickListener { finish() }    // closes this Activity
```

Other getters **: `getIntExtra("key", defaultValue)`, `getDoubleExtra("key", 0.0)`, `getBooleanExtra("key", false)`. Passing data enables personalisation of the next screen, sharing context between Activities and dynamic UI.

6. ScrollView **

A **ScrollView** makes content scrollable when it is taller than the screen (for example a long registration form or an article).

Key rule: a ScrollView can have only ONE direct child. So you put a LinearLayout (or another layout) inside it, and place all your views inside that layout.

```
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <EditText android:hint="Name" ... />
        <EditText android:hint="Email" ... />
        <EditText android:hint="Address" ... />
        <Button android:text="Submit" ... />
    </LinearLayout>
</ScrollView>
```

- The child's height should be `wrap_content` (it grows with its content; the ScrollView scrolls it).
- `HorizontalScrollView` scrolls sideways. `NestedScrollView` is the newer version that works with Material scrolling effects.
- ScrollView keeps **all** its child views in memory. That is fine for a form, but bad for long data lists; use RecyclerView for lists. Do not put a RecyclerView or ListView inside a ScrollView (two scrolling containers fight each other).

7. Adapters and ListView

What is an Adapter?

An **Adapter** is a bridge between a **data source** (a List or ArrayList) and a **list view** (ListView, Spinner, RecyclerView). It takes each data item and turns it into a row view the user sees.

Data (list of items) → Adapter (creates and fills row views) → ListView / RecyclerView (displays the rows)

Adapter	Used with	Notes
`ArrayAdapter`	ListView, Spinner	Simplest; shows one text per row using a built-in row layout
Custom adapter (`BaseAdapter`)	ListView with custom rows (image + two texts)	You write `getView()` yourself
`RecyclerView.Adapter`	RecyclerView	Uses ViewHolder; must override 3 methods

ListView with ArrayAdapter **

ListView is the older, simpler way to show a vertical scrolling list.

```
<ListView
    android:id="@+id/listView"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

```
val fruits = listOf("Apple", "Banana", "Orange")
val listView = findViewById<ListView>(R.id.listView)

val adapter = ArrayAdapter(this, android.R.layout.simple_list_item_1, fruits)
listView.adapter = adapter

// Row click: show which item was tapped
listView.setOnItemClickListener { _, _, position, _ ->
    Toast.makeText(this, "Clicked ${fruits[position]}", Toast.LENGTH_SHORT).show()
}
```

`ArrayAdapter(context, rowLayout, data)`: `android.R.layout.simple_list_item_1` is a built-in Android layout containing one TextView.

The same adapter fills a **Spinner** (drop-down): `spinner.adapter = ArrayAdapter(this, android.R.layout.simple_spinner_dropdown_item, movies)` and read the choice with `spinner.selectedItem.toString()`.

Custom adapter for ListView ** (when each row has more than one text)

```
class StudentListAdapter(private val context: Context, private val students: List<Student>)
    : BaseAdapter() {
    override fun getCount() = students.size                  // number of rows
    override fun getItem(position: Int) = students[position] // data for a row
    override fun getItemId(position: Int) = position.toLong()
    override fun getView(position: Int, convertView: View?, parent: ViewGroup): View {
        // reuse the old row (convertView) if Android gives one, otherwise inflate a new one
        val row = convertView ?: LayoutInflater.from(context)
            .inflate(R.layout.item_student, parent, false)
        row.findViewById<TextView>(R.id.tvName).text = students[position].name
        row.findViewById<TextView>(R.id.tvRegNo).text = students[position].regNo
        return row
    }
}
```

8. RecyclerView

What and why

RecyclerView is a ViewGroup for efficiently displaying **large, scrollable sets of data**. It is the modern replacement for ListView and GridView (part of AndroidX: `androidx.recyclerview.widget.RecyclerView`).

Recycling: it only creates enough row views to fill the screen plus a few extra. When a row scrolls off one edge, its view is **not destroyed**; it goes into a recycle pool and is **re-bound with new data** when a new item scrolls in from the other edge. Result: better performance, smoother scrolling, less memory and lower power use, even with thousands of items.

The four working parts (must memorise)

Part	Role
RecyclerView	The scrolling container placed in the layout XML
Adapter	Creates ViewHolders and binds (puts) data into them
ViewHolder	Holds one row's view and caches references to its child views, so `findViewById` is called only once per row, not every time it is reused
LayoutManager	Decides how rows are arranged on screen

The three Adapter methods (must memorise)

Method	What it does	When it is called
`onCreateViewHolder(parent, viewType)`	**Inflates** the row XML and returns a new ViewHolder (no data yet)	Only when a new row view is needed
`onBindViewHolder(holder, position)`	**Fills** the ViewHolder's views with the data at `position`	Every time a row appears on screen (including reused rows)
`getItemCount()`	Returns the **total number** of items	Whenever RecyclerView needs the size

LayoutManagers

LayoutManager	Arrangement
`LinearLayoutManager(this)`	Single vertical list (or horizontal with `LinearLayoutManager(this, LinearLayoutManager.HORIZONTAL, false)`)
`GridLayoutManager(this, 2)`	Grid of equal-size cells; the number is the span (column) count
`StaggeredGridLayoutManager(2, StaggeredGridLayoutManager.VERTICAL)`	Grid with different item heights (Pinterest style)

Common bug: if you forget to set a LayoutManager, the RecyclerView shows **nothing**. Set it in code or with `app:layoutManager` in XML.

Steps to implement RecyclerView

1. Add the dependency: `implementation("androidx.recyclerview:recyclerview:1.3.2")`
2. Add `<RecyclerView>` to the Activity layout.
3. Design one row layout (often a MaterialCardView).
4. Create a model (data) class.
5. Create the Adapter with a nested ViewHolder and the 3 methods.
6. In the Activity: find the RecyclerView, set the LayoutManager, set the Adapter.

Full example: Student Directory (homework scenario: CardView rows + Toast on click)

Model class (a Kotlin data class holding one student's data):

```
data class Student(val name: String, val regNo: String, val department: String)
```

activity_main.xml

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recyclerView"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

item_student.xml (one row, wrapped in a Material card)

```
<com.google.android.material.card.MaterialCardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    app:cardCornerRadius="12dp"
    app:cardElevation="4dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">
        <TextView android:id="@+id/tvName"   android:layout_width="wrap_content"
android:layout_height="wrap_content" android:textSize="18sp"/>
        <TextView android:id="@+id/tvRegNo"  android:layout_width="wrap_content"
android:layout_height="wrap_content"/>
        <TextView android:id="@+id/tvDept"   android:layout_width="wrap_content"
android:layout_height="wrap_content"/>
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
```

StudentAdapter.kt

```
class StudentAdapter(private val students: List<Student>) :
    RecyclerView.Adapter<StudentAdapter.StudentViewHolder>() {

    // ViewHolder: finds the row's views ONCE and keeps the references
    class StudentViewHolder(view: View) : RecyclerView.ViewHolder(view) {
        val name: TextView = view.findViewById(R.id.tvName)
        val regNo: TextView = view.findViewById(R.id.tvRegNo)
        val dept: TextView = view.findViewById(R.id.tvDept)
    }

    // 1. Create a new row (inflate the XML)
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): StudentViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_student, parent, false)
        return StudentViewHolder(view)
    }

    // 2. Put the data for this position into the row
    override fun onBindViewHolder(holder: StudentViewHolder, position: Int) {
        val student = students[position]
        holder.name.text = student.name
        holder.regNo.text = student.regNo
        holder.dept.text = student.department

        // click on a row -> Toast with the student's name
        holder.itemView.setOnClickListener {
            Toast.makeText(holder.itemView.context, student.name, Toast.LENGTH_SHORT).show()
        }
    }

    // 3. How many rows
    override fun getItemCount(): Int = students.size
}
```

MainActivity.kt

```
val recyclerView = findViewById<RecyclerView>(R.id.recyclerView)
recyclerView.layoutManager = LinearLayoutManager(this)       // REQUIRED

val students = arrayListOf(
    Student("Nimal Perera", "IT21001", "IT"),
    Student("Kasuni Silva", "IT21002", "IM")
    // ... at least 10
)
recyclerView.adapter = StudentAdapter(students)
```

Explain `inflate(R.layout.item_student, parent, false)`: converts the row XML into a View object; `parent` gives correct layout sizes; `false` means “do not attach it yet” (RecyclerView attaches it itself).

RecyclerView has no built-in click listener, so clicks are set inside `onBindViewHolder` ** (as above) or inside the ViewHolder.

Updating the list later

- `notifyDataSetChanged()`: rebinds the whole list; simple but slow and disables animations.
- Better, targeted calls: `notifyItemInserted(position)`, `notifyItemRemoved(position)`, `notifyItemChanged(position)`.
- Best: `ListAdapter` + `DiffUtil`, which calculates the minimal changes automatically and you call `adapter.submitList(newList)`.

ListView vs RecyclerView

Aspect	ListView (legacy)	RecyclerView
View reuse	Manual, via `convertView`	Automatic, enforced by framework
ViewHolder pattern	Optional	Required (built into the API)
Layout	Vertical list only	Linear, Grid, Staggered Grid via LayoutManager
Item animations	Manual	Built-in (ItemAnimator)
Dividers / decoration	Manual drawing	ItemDecoration API
Click handling	Built-in `setOnItemClickListener`	Must be wired manually
Code	Less code, simpler	More boilerplate, more flexible
Best for	Small, simple lists	Large or dynamic data

RecyclerView advantages: handles very large datasets efficiently; lists, grids and staggered grids; lower power use; extensible (ItemDecoration, ItemAnimator). **Disadvantages:** more boilerplate; no built-in click listener; must understand several classes together.

ScrollView vs RecyclerView: ScrollView scrolls one fixed set of views and keeps them all in memory (forms). RecyclerView displays repeated, data-driven items and recycles them (lists of data).

Newer note: in Jetpack Compose the equivalent is `LazyColumn` / `LazyRow`.

9. Fragments

Why Fragments? (lecture argument)

If every destination (Home, Users, Settings) is a full Activity, you get: heavy memory use, complex navigation between screens, and the toolbar / drawer / bottom navigation duplicated on every screen.

A **Fragment** represents a **portion of the UI** (not a whole screen). It lives **inside an Activity** and has **its own lifecycle and layout**.

Fragments solve the problem by:

- allowing multiple “screens” to live inside **one Activity**,
- supporting dynamic UI changes **without recreating the Activity**,
- working naturally with the **navigation drawer** and **bottom navigation** (each menu item swaps a Fragment into one container).

Used in practice for: navigation drawer content panels, bottom navigation sections, tabs, multi-pane tablet layouts. Fragments are **reusable** (the same Fragment can appear in different Activities).

Activity	Fragment **
A full screen; entry point of a screen	A reusable part of a screen
Can exist on its own	Must be hosted inside an Activity
Declared in AndroidManifest.xml	Not declared in the manifest
Heavier; switching uses Intents	Lighter; switching uses FragmentManager transactions

Fragment code **

1. Fragment layout `fragment_home.xml` (any normal layout, for example a TextView).

2. Fragment class:

```
class HomeFragment : Fragment() {
    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?, savedInstanceState: Bundle?
    ): View? {
        // turn the fragment's XML into a View and return it
        return inflater.inflate(R.layout.fragment_home, container, false)
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        // find views through 'view', and use requireContext() instead of 'this'
        val btn = view.findViewById<Button>(R.id.btnHello)
        btn.setOnClickListener {
            Toast.makeText(requireContext(), "Hello from Home", Toast.LENGTH_SHORT).show()
        }
    }
}
```

3. Container in the Activity layout (FrameLayout or FragmentContainerView):

```
<FrameLayout
    android:id="@+id/fragmentContainer"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1" />
```

4. Show / swap a Fragment (fragment transaction):

```
supportFragmentManager.beginTransaction()
    .replace(R.id.fragmentContainer, HomeFragment())
    .addToBackStack(null)        // optional: Back button returns to previous fragment
    .commit()                    // nothing happens without commit()
```

`add()` puts a fragment on top; `replace()` removes the current one and shows the new one; `remove()` removes one.

5. With Bottom Navigation:

```
bottomNavigation.setOnItemSelectedListener { item ->
    val fragment = when (item.itemId) {
        R.id.nav_home  -> HomeFragment()
        R.id.nav_users -> UsersFragment()
        else -> return@setOnItemSelectedListener false
    }
    supportFragmentManager.beginTransaction()
        .replace(R.id.fragmentContainer, fragment).commit()
    true
}
```

Fragment lifecycle (main methods): `onAttach` → `onCreate` → **`onCreateView`** (create the UI) → `onViewCreated` → `onStart` → `onResume` → `onPause` → `onStop` → `onDestroyView` → `onDestroy` → `onDetach`.

10. Navigation patterns

Pattern	Best for
Top App Bar (Toolbar)	Shows screen title, action items (logout, search); consistent branding. Sits above the other patterns.
Bottom Navigation	**3 to 5** frequently used top-level destinations; the most common modern pattern; thumb-friendly
Tabs	Switching between related views of the same data
Navigation Drawer	**Many** destinations (5 or more), including rarely used ones; slides in from the left

Toolbar with a menu (lecture code):

```
<com.google.android.material.appbar.MaterialToolbar
    android:id="@+id/toolbar"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:title="Home"
    app:menu="@menu/top_menu"/>

<!-- res/menu/top_menu.xml -->
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/action_logout" android:title="Logout"
          android:showAsAction="always"/>
</menu>
```

```
toolbar.setOnMenuItemClickListener {
    if (it.itemId == R.id.action_logout) { showLogoutDialog(); true } else false
}
```

AlertDialog to confirm a destructive action (so one accidental tap does not log the user out):

```
AlertDialog.Builder(this)
    .setTitle("Logout")
    .setMessage("Do you want to logout?")
    .setPositiveButton("Yes") { _, _ -> finish() }   // runs only when Yes is tapped
    .setNegativeButton("No", null)                    // null = just close the dialog
    .show()
```

Dialogs are used to confirm important actions, warn before irreversible actions and show short information without leaving the screen.

Bottom Navigation XML:

```
<com.google.android.material.bottomnavigation.BottomNavigationView
    android:id="@+id/bottomNavigation"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:menu="@menu/bottom_menu"/>
<!-- res/menu/bottom_menu.xml: <item android:id="@+id/nav_home" android:title="Home"/> ... -->
```

Navigation Drawer structure:

```
<androidx.drawerlayout.widget.DrawerLayout ...>
    <FrameLayout android:id="@+id/container" .../>          <!-- main content: fragments swap here -->
    <com.google.android.material.navigation.NavigationView
        app:menu="@menu/drawer_menu"
        android:layout_gravity="start"/>                   <!-- the sliding menu on the left -->
</androidx.drawerlayout.widget.DrawerLayout>
```

11. Material Design

Definition

Material Design is Google's **design system**: guidelines, reusable components and best practices for building Android apps that are **consistent, easy to use, responsive across screen sizes, accessible and visually appealing**. Launched in **2014**. **Material Design 3 ("Material You")** came in **2021** with Android 12; its signature feature is **dynamic color** (the app's colour palette is taken from the user's wallpaper, Android 12 / API 31+).

The three core principles (anchor every answer on these)

Principle	Meaning	Example
1. Material is the metaphor / Material as a surface	Components behave like physical sheets of paper with height (**elevation**), shadows and clear edges, so users can see what is clickable or on top	Cards look raised; buttons look pressable
2. Bold, graphic and intentional design	Deliberate colour, large clear typography, icons, white space, so important actions and errors stand out	One strong primary button; red error text
3. Motion provides meaning	Animations guide attention and confirm cause and effect; smooth, purposeful, not distracting (about 150 to 300 ms)	Button ripple, screen transitions, loading indicators

Material components

Component	Purpose / guidance
MaterialButton	4 styles by emphasis: **Filled** (highest, the one main action), **Elevated**, **Outlined**, **Text** (lowest, for “Cancel”)
TextInputLayout + TextInputEditText	Text field with floating label, inline error message, helper text, password toggle. A plain EditText has no built-in error handling or visual feedback
MaterialCardView	Groups related information on one surface with rounded corners and elevation
Floating Action Button (FAB)	The single primary action of a screen (add, compose). Only **one** per screen, bottom-right, about 16dp margin
MaterialToolbar / App Bar	Title, navigation icon, actions at the top
BottomNavigationView	3 to 5 top-level sections
NavigationView (drawer)	Side menu for 5 or more destinations
Snackbar	Short message with an optional action

TextInputLayout (lecture code):

```
<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/tilEmail"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Email Address">
    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/etEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textEmailAddress"/>
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Password"
    app:endIconMode="password_toggle">          <!-- show/hide eye icon, no Kotlin needed -->
    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/etPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPassword"/>
</com.google.android.material.textfield.TextInputLayout>
```

Show an inline error from Kotlin **: `tilEmail.error = "Please enter a valid email"` (clear it with `tilEmail.error = null`).

MaterialButton and MaterialCardView:

```
<com.google.android.material.button.MaterialButton
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Submit"/>

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardCornerRadius="12dp"
    app:cardElevation="4dp"> ... </com.google.android.material.card.MaterialCardView>
```

FAB **:

```
<com.google.android.material.floatingactionbutton.FloatingActionButton
    android:id="@+id/fabAdd"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom|end"
    android:layout_margin="16dp"
    android:src="@drawable/ic_add"/>
```

Material foundations

- **Theme:** a set of styles (colours, typography, shapes) applied across all screens. Example parent `Theme.Material3.DayNight.NoActionBar` gives automatic light and dark mode.
- **Colour system:** limited palette with roles: **Primary** (main brand colour), **Secondary** (accent), **Tertiary** (M3 accent), **Background**, **Surface** (cards, sheets, dialogs), **Error** (validation). M3 builds each role as a tonal palette.
- **Typography:** a type scale (Headline, Title, Body, Label, Caption) for consistent sizes. Body text about 14sp or more; use sp so text scales with user settings.
- **Icons:** Material Symbols in Outlined, Rounded and Sharp styles.
- **Elevation:** higher elevation = closer to the user = bigger shadow. Typical: card 1 to 3dp, app bar 0 to 4dp, FAB about 6dp.
- **Responsive design:** works on phones, tablets, foldables, Chromebooks; design for window size classes (Compact, Medium, Expanded).

Advantages	Disadvantages
Consistent, attractive, modern UI	Full component set can be challenging for beginners
Easy navigation, responsive across devices	Advanced customisation can get complex
Rich library of reusable, tested components	Apps using it may look visually similar
Better accessibility and faster UI development	

Real apps: Gmail (FAB, cards, drawer), Google Drive (cards, bottom nav), Google Maps (app bar, dynamic colour), Google Photos (grid, bottom nav).

Responsive layout (lecture)

A responsive UI must work on every screen size, support orientation changes and remain accessible at any density. Techniques: prefer ConstraintLayout over fixed positioning, avoid fixed pixel sizes, use dp for spacing and sp for text, test on multiple device configurations.

12. UI/UX flow design

The design process (three stages, revisit earlier stages after feedback)

1. Wireframe → 2. Prototype → 3. Usability test

Stage	What it is	Purpose
Wireframe	**Low-fidelity, grayscale** layout showing structure and content placement, **without** colours, fonts or icons	Focus on structure and flow; quick to change before any code; team agrees on layout early
Prototype	Wireframes **connected** with clickable hotspots to **simulate the flow** between screens (made in **Figma** in this module)	Review navigation before writing code
Usability test	Observe real users completing tasks with the design	Find friction and problems

Finding a design problem at the **wireframe stage is much cheaper** than finding it after implementation.

Layout and interaction principles

- Support every orientation and screen size (portrait/landscape, phone/tablet).
- Respect safe areas: keep content clear of system bars, display cutouts and the keyboard.
- Keep primary actions reachable (for example a FAB, or a bottom button).
- Group related content using cards and sections.
- Do not overload one screen; split complex tasks into steps.
- Keep spacing and alignment consistent.

Usability heuristics (adapted from Jakob Nielsen's 10)

Heuristic	Meaning
Visibility of system status	Always show loading, progress, success or error states
Match real-world conventions	Use icons and words users already know
User control and freedom	Clear Back or Cancel on every screen
Consistency and standards	Same button style and placement across screens
Recognition over recall	Show options instead of making users remember them
Error prevention	Confirm before destructive actions (delete, logout), validate input

Small-screen design numbers (memorise)

Rule	Value
Minimum touch target	**48dp × 48dp**
Legible body text	Not smaller than **12sp** (lecture); Material guide recommends about 14sp; always use **sp**
Colour contrast	At least **4.5 : 1** between text and background
Reach zone	Primary actions **low** on the screen (easy thumb reach); rare actions near the top (hard to reach)

QuickCart case examples: "Pay Now" at top-right is outside the thumb's reach zone → move it to the bottom. Small, close +/- buttons break the 48dp touch target rule → enlarge and space them. A generic "Error!" message does not say which field is wrong → show an inline message under the field ("Please enter a valid email address"). "Create Account" gives no feedback after tapping → show a loading state (visibility of system status). Too many options on Settings → group them (do not overload one screen).

13. Practice questions with short model answers

Why use RecyclerView instead of ListView for a list of 1,000 students? (2 marks) RecyclerView recycles row views through the ViewHolder pattern, so only the visible rows are created. This uses less memory and scrolls smoothly; it also supports grid layouts and built-in animations.

Name the four components of RecyclerView and the role of each. RecyclerView (container), Adapter (creates and binds rows), ViewHolder (caches one row's views), LayoutManager (arranges rows).

Which method fills a row with data? `onBindViewHolder(holder, position)`.

A RecyclerView shows no data although the adapter is set. Why? The LayoutManager was not set.

Write code to display “Saved” for a short time. `Toast.makeText(this, "Saved", Toast.LENGTH_SHORT).show()`

Why does a ScrollView contain a LinearLayout? A ScrollView can host only one direct child, so the LinearLayout groups all the views into that one child.

ScrollView or RecyclerView for a long registration form? Justify. ScrollView, because the form is a fixed set of different input views, not repeated data items.

What is an Adapter? A bridge that takes data from a list and creates the row views shown in a ListView, Spinner or RecyclerView.

Why use Fragments? They let several reusable UI parts live in one Activity, so content can change (bottom nav, drawer) without starting new Activities, saving memory and avoiding duplicated toolbars and menus.

Explicit vs implicit Intent? Explicit names the target class (opening HomeActivity); implicit describes an action and lets Android choose an app (opening a web page).

State the three principles of Material Design. Material as a surface (elevation, shadows), bold graphic intentional design (colour, typography, icons), meaningful motion (animations that guide attention).

Why use TextInputLayout instead of EditText? It adds a floating label, inline error messages, helper text and a password toggle, giving clear feedback (Material Design).

Bottom navigation or navigation drawer for an app with 4 main sections? Justify. Bottom navigation, because it is designed for 3 to 5 frequently used destinations and is easy to reach with the thumb.

Difference between wireframe and prototype? A wireframe is a static low-fidelity sketch of one screen’s structure; a prototype links screens with clickable hotspots to simulate the flow.

dp vs sp? dp is for sizes and spacing; sp is for text and also scales with the user’s font-size setting.

What is the minimum touch target size? 48dp × 48dp.